

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 2

Выполнила:

Савченко Анастасия

Группа К3341, Поток БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

На основе документа, сформированного в рамках ДЗ4, реализовать разделение монолитного бэкенд-приложения бронирования столиков на микросервисы с соблюдением концепции database-per-service, настроить межсервисное взаимодействие и API Gateway.

Ход работы

1. Структура микросервисов

На основе архитектурного проекта из ДЗ4 монолит разделён на три независимых микросервиса и API Gateway. Каждый сервис имеет собственную базу данных, набор моделей и независимый запуск.

Микросервис	Порт	База данных	Ответственность
auth-service	8001	auth_db	Регистрация, аутентификация, профили пользователей
restaurant-service	8002	restaurant_db	Каталог ресторанов, кухни, меню, столики, фотографии
booking-service	8003	booking_db	Бронирования, отзывы, статистика
api-gateway	3000	—	Единая точка входа, прокси запросов к сервисам

2. Разделение базы данных

Каждый сервис использует собственную базу данных PostgreSQL. Внешние ключи между базами заменены на идентификаторы без ограничений целостности. Ссылочная целостность проверяется на уровне приложения через внутренние HTTP-запросы.

auth_db содержит таблицу Users (7 полей: первичный ключ, email, хэш пароля, имя, телефон, даты создания и обновления).

restaurant_db содержит шесть таблиц: Restaurants (центральная), Cuisines (справочник), RestaurantCuisine (ассоциативная M:N), Tables

(столики), MenuItems (блюда), RestaurantPhotos (фотографии). Все зависимые таблицы связаны с Restaurants через внешний ключ restaurant_id.

booking_db содержит две таблицы: Bookings (бронирования с полями даты, времени, статуса и ссылками на пользователя, ресторан и столик) и Reviews (отзывы с оценкой и ссылками на пользователя и ресторан). Ссылки на сущности из других сервисов хранятся как обычные числовые поля.

3. Структура каждого сервиса

Все три сервиса построены по единому шаблону:

service/

- |— package.json # зависимости и скрипты
- |— tsconfig.json # настройки TypeScript с декораторами
- |— seed.ts # заполнение БД тестовыми данными
- |— src/
 - |— models/ # entity TypeORM (только таблицы сервиса)
 - |— data-source.ts # подключение к своей БД
 - |— index.ts # эндпоинты, запуск сервера

auth-service содержит одну модель (User).
restaurant-service содержит шесть моделей (Restaurant, Cuisine, RestaurantCuisine, Table, MenuItem, RestaurantPhoto).
booking-service содержит две модели (Booking, Review) без декораторов связей к другим сервисам.

4. Внешние эндпоинты

Каждый сервис предоставляет свою часть публичного API, соответствующую спецификации из Д32:

auth-service (8001):

- POST /api/v1/auth/register — регистрация с хэшированием пароля
- POST /api/v1/auth/login — вход с выдачей JWT-токенов

- POST /api/v1/auth/refresh — обновление токенов
- GET /api/v1/users/me — профиль текущего пользователя (защищён)
- PATCH /api/v1/users/me — обновление профиля restaurant-service (8002):
- GET /api/v1/cuisines — список кухонь
- GET /api/v1/restaurants — список ресторанов с фильтрацией и пагинацией
- GET /api/v1/restaurants/:id — детали ресторана
- GET/POST/PATCH/DELETE для блюд (/api/v1/restaurants/:id/dishes, /api/v1/dishes/:id)
- GET /api/v1/restaurants/:id/tables и /api/v1/restaurants/:id/tables/available
- GET/POST/DELETE для фотографий
- booking-service (8003):
- GET /api/v1/reservations — история бронирований (защищён)
- POST /api/v1/reservations — создание брони с проверкой занятости
- PATCH /api/v1/reservations/:id/cancel — отмена брони
- GET/POST/PATCH/DELETE для отзывов

5. Внутренние эндпоинты (межсервисное взаимодействие)

Для обеспечения проверок существования сущностей между сервисами реализованы внутренние эндпоинты, недоступные через API Gateway:

auth-service:

- GET /api/internal/users/{user_id}/exists — возвращает { exists: true, user: {...} } или 404

restaurant-service:

— GET /api/internal/restaurants/{restaurant_id}/exists — проверка ресторана

— GET /api/internal/tables/{table_id}/exists?restaurant_id=... — проверка столика с дополнительной проверкой принадлежности ресторану booking-service:

— GET /api/internal/restaurants/{restaurant_id}/stats — возвращает { rating_avg, reviews_count }

6. Межсервисное взаимодействие

При создании брони booking-service выполняет два внутренних HTTP-запроса:

1. К restaurant-service — проверка существования ресторана (GET /api/internal/restaurants/{id}/exists)

2. К restaurant-service — проверка существования столика и его принадлежности ресторану (GET /api/internal/tables/{id}/exists?restaurant_id=...)

При создании отзыва booking-service проверяет существование ресторана через аналогичный запрос к restaurant-service.

В обратную сторону restaurant-service может получать статистику ресторана из booking-service через запрос к /api/internal/restaurants/{id}/stats (реализовано в виде эндпоинта, интеграция в выдачу каталога запланирована).

Все внутренние запросы выполняются синхронно через HTTP (fetch API). При недоступности сервиса возвращается ошибка.

7. API Gateway

API Gateway реализован на Express (порт 3000) и выполняет роль единой точки входа для клиентов. Все запросы к /api/v1/* проксируются на соответствующие микросервисы:

— /api/v1/auth/* и /api/v1/users/* → auth-service (8001)

- /api/v1/restaurants/* (кроме reviews) → restaurant-service (8002)
- /api/v1/restaurants/*/reviews → booking-service (8003)
- /api/v1/cuisines, /api/v1/dishes, /api/v1/photos → restaurant-service (8002)
- /api/v1/reservations → booking-service (8003)
- Gateway пробрасывает HTTP-метод, заголовки (включая Authorization) и тело запроса. При получении не-JSON ответа корректно обрабатывает его без падения.

8. Тестирование

Проведено сквозное тестирование через API Gateway (порт 3000):

- Регистрация нового пользователя — успешно (201)
- Вход в систему — успешно (200), получен JWT-токен
- Создание бронирования — успешно (201), межсервисная проверка ресторана и столика отработала
- Создание отзыва — успешно (201)
- Проверка истории бронирований — успешно (200)
- Бронь с несуществующим рестораном — ошибка 404 (booking-service проверил через restaurant-service)
- Отзыв к несуществующему ресторану — ошибка 404
- Внутренний эндпоинт статистики — возвращает корректные данные
- Межсервисное взаимодействие работает корректно.

Вывод

В результате выполнения лабораторной работы монолитное бэкенд-приложение успешно разделено на три независимых микросервиса: auth-service, restaurant-service и booking-service. Каждый сервис использует собственную базу данных (database-per-service). Реализовано синхронное

HTTP-взаимодействие между сервисами через внутренние эндпоинты для проверки целостности данных. API Gateway обеспечивает единую точку входа для клиентов. Проведено тестирование, подтверждающее корректную работу всех сервисов и межсервисных проверок. Архитектура готова к дальнейшему развитию: добавлению асинхронного взаимодействия через RabbitMQ (Д35) и контейнеризации (ЛР3).